

Learning Shooting and Goalkeeping Skills for the Unitree G1 Humanoid

Team 12
CS2810 Embodied AI, Spring 2026

Abstract

We develop reinforcement-learning soccer skills for the 29-DoF Unitree G1 humanoid in the mjlal MuJoCo simulator, covering the project’s two tasks: *shooting* (approach and kick a stationary ball into the goal) and *goalkeeping* (intercept a ball launched on a parabolic trajectory). For the shooter we build a six-stage progressive perception–action pipeline on a single shared LSTM policy, reaching a **99.96%** goal rate over 3×10^4 trials (Phase-1 score $\approx 29.9/30$) with 12–16 m/s ball speeds, and deploy it behind a multi-strategy API server whose goalkeeper-aware policy reads the opponent and shoots to the far side. For the goalkeeper we treat diving saves – unreachable by reward-shaped exploration and suppressed by any upright prior – as a staged pipeline: distil a diving base from a reference teacher, use a per-scenario CEM repair oracle to synthesize beyond-teacher saves, train a bounded ballistic-residual specialist per ball region, and assemble them into a six-way mixture of experts routed by an analytic ballistic-crossing gate with a left/right symmetry mirror. This lifts the block rate from a 66.9% distilled base, through an 81.7% single network, to a deployed mixture at $\approx 93.8\%$ (a 16,384-trial estimate; best observed $475/500 = 95.0\%$ on the official 500-trial eval). We report both tasks honestly, including negative results and a failure-replay path for further gains, and detail the API observation computation each policy performs server-side from raw MuJoCo state.

Contents

1	Introduction	1
	Part A: Goalkeeper	2
2	Task and the core difficulty	2
3	Method: a staged pipeline culminating in a region mixture-of-experts	2
4	Phase 1 result and analysis	4
5	Secondary experiments and negative results	5
6	Phase 2: deployment behind the competition API	5
	Part B: Shooter	7
7	Overview	7
8	Training pipeline: six-stage progressive training	7
9	Model architecture and observation space	9
10	Reward design evolution	10

11 Phase 1 evaluation	11
12 Phase 2: competition strategy and API server	11
13 Motion performance analysis	13
14 Engineering decisions and lessons	14

1 Introduction

The project develops two humanoid soccer skills on the Unitree G1 (29 DoF) in the mjlab MuJoCo-based physics simulation [7], each controlled at 50 Hz:

- **Shooting:** the robot approaches a stationary ball and kicks it through the goal frame. This is a perception-guided motor skill that needs a motion-tracking prior and ball-trajectory control.
- **Goalkeeping:** the robot intercepts an incoming ball launched on a parabolic trajectory toward a $3.0\text{ m} \times 1.8\text{ m}$ goal. This is a reactive interception task requiring rapid end-effector positioning and ball trajectory estimation.

The grading has two phases. **Phase 1** evaluates each single-agent policy over 50 trials with the official `eval_naive_shooter.py` / `eval_naive_goalkeeper.py` scripts, scored by the linear band score = $\max(0, (\text{success} - 0.8)/0.2) \times 30$ (zero below 80%, full at 100%). **Phase 2** is a centrally run cross-evaluation tournament in which each team serves its policy behind a standard `/act//reset` HTTP API and `compete.py` queries both teams at every simulation step. Recent humanoid-soccer work informs the environment and training design [1, 2].

This report is organized by task. Part 1 (Sections 2–6) covers the goalkeeper; Part 6 (Sections 7–14) covers the shooter. Each part reports its Phase-1 results with analysis and its Phase-2 design, including how observations are computed from raw state.

Part A Goalkeeper

The deliverable is the `keeper-lyk` branch, whose final goalkeeper is a **six-way region mixture-of-experts** (the MoE6 bundle `goalkeeper_moe6_hard3_default.pt`): one bounded ballistic-residual specialist per ball region, routed at inference by an analytic ballistic-crossing gate. On the official evaluation it reaches a best-observed $475/500 = 95.0\%$ block rate, with a more stable $\approx 93.8\%$ over 16,384 trials – above the 91% sibling MoE on the `keeper-qbs` branch, the 81.7% single repaired network it is built from, and the $\approx 72.5\%$ reference teacher.

2 Task and the core difficulty

The goalkeeper must intercept a ball launched on a parabolic trajectory toward a full $3\text{ m} \times 1.8\text{ m}$ goal, controlling the G1 at 50 Hz. A save is credited only if the ball never crosses the goal plane within the frame. Crucially, the diving saves required to defend the corners necessarily terminate with the robot on the ground.

This single property reshapes the entire problem. Any learning signal that penalizes falling simultaneously penalizes the dives that produce saves, so the conventional “remain upright” inductive bias is actively counterproductive; the evaluation reflects this by not treating a fall as a failure. The consequence is that **the required interception behavior is not reachable by reward-shaped exploration** – under a hand-designed reward, a humanoid does not stumble into a committed, directional dive through random search. We verified

this empirically: trained from scratch with PPO [3] under a range of shaped rewards, the policy reliably learns to stand and to satisfy the proxy terms, but its block rate saturates between 3% and 33%, with the high-ball corners frozen near zero. A weak reward yields passive standing ($\approx 33\%$, barely above the do-nothing baseline); a strong one induces an uncontrolled lunge that topples without intercepting.

The central design question is therefore not “which reward yields a goalkeeper” but rather **how to inject a diving prior, and how to subsequently improve upon it against the true objective**. The remainder of the method follows from this premise.

A second observation establishes the target. A per-region diagnosis of the Humanoid Goalkeeper of Ren et al. [2] – whose checkpoint serves as our teacher – shows that it blocks only $\approx 72.5\%$ on our benchmark, with failures concentrated almost entirely in the high balls (low balls $\approx 95\%$, the upper corners $\approx 40\text{--}60\%$). This is doubly informative: competent behavior still leaves the corners largely undefended, and it implies that **pure imitation of Ren et al. [2] cannot exceed $\approx 72.5\%$** . To do better, we must *synthesize* high-ball saving behavior that the teacher does not itself possess.

3 Method: a staged pipeline culminating in a region mixture-of-experts

Because the diving prior cannot be acquired from scratch and imitation of Ren et al. [2] is upper-bounded at $\approx 72.5\%$, no single training procedure is sufficient. We adopt a **staged pipeline** in which each stage overcomes the failure mode of the preceding one: distil a diving base (Stage A); *prove* with an offline optimizer that the conceded balls are physically saveable and fold those saves into one network (Stages B–C); *specialize* a bounded corrective head per ball region (Stage D); and route the specialists with an analytic gate into the deployed mixture (Stage E). The deliverable is the six-way mixture bundle `goalkeeper_moe6_hard3_default.pt`.

3.1 Stage A — Distillation: injecting the diving prior

We behavior-clone the teacher policy of Ren et al. [2] into a trainable network of our own design, since the released checkpoint is inference-only under the current RL framework [8] and cannot itself be optimized. The procedure follows DAgger [4]: the student is rolled out, the teacher labels the visited states, and the aggregated dataset is cloned. This yields a competent, reactive *diving* base policy – the 960-D-input MLP checkpoint `goalkeeper_distilled_v3.pt`. It does not surpass the teacher – imitation cannot – scoring $\approx 74\%$ over 100 trials and $\approx 66.9\%$ on the harder vectorized benchmark used in the rest of this part. It nevertheless provides a trainable policy that already executes dives, the precondition for every subsequent stage.

3.2 Stage B — Repair oracle: proving the hard balls are saveable

To exceed the base policy we must save high balls that it concedes. We construct a **per-scenario trajectory optimizer**: for the frozen base policy and a fixed incoming ball, it searches – using the cross-entropy method, CEM [6, 5], a sampling-based optimizer – an open-loop *residual* action sequence added to the base policy’s actions. The search minimizes a dense cost combining the true concede outcome with the distance from the nearest blocking limb to the ball’s predicted plane-crossing point. This optimizer **demonstrates that the hard balls are physically saveable** at a per-scenario save rate far above the base policy, establishing that control *authority* rather than kinematic reach was the limiting factor. The optimizer is single-agent simulation optimization, not a learned policy – it is used as a *data generator*, not as the deliverable.

3.3 Stage C — Distilling the repaired saves into a single network

The repaired residuals are per-scenario and open-loop; they cannot be deployed directly. We therefore **collect the repaired (observation, action) pairs from trajectories that the oracle turns into successful**

saves, and distil them back into a native MLP with the same architecture as the base. The distilled checkpoint, `model_repaired_lyk.pt`, reaches **81.7%** – a +14.8-point gain over the 66.9% base. This single reactive network is strong but still pays the single-policy trade-off (improving one corner regresses another); it becomes the *shared base* that the region specialists of Stage D each refine. Two collection choices proved essential, each of which silently degraded the result when violated:

- *Demonstrations must be evaluation-matched.* Forcing a scenario after the environment reset leaves stale ball-history frames in the observation, training the policy on a distribution it never encounters at evaluation. The scenario must be imposed *through* the reset so that the observation history is consistent with the deployed keeper observation/history layout.
- *Collect a stable save window, not the post-save fall.* A naive collection learns the high-ball save together with the subsequent topple-and-flail, so the deployed policy blocks but then loses balance after contact. The oracle residual is faded back to the base policy after the save window, and distillation keeps **only the frames around the keeper-plane crossing instant** (a short pre/post window), so the student learns the interception rather than the fall.

A **region-weighted, multi-GPU collection** path supports the same pipeline: one repair worker per GPU, with region weights that oversample low/mid balls (regions 4, 5 low, 0, 1 mid, 2, 3 upper) to keep the distilled policy from overfitting to the dramatic high-ball jumps at the expense of routine balls.

3.4 Stage D — Per-region ballistic-residual specialists

A single reactive network cannot simultaneously hold six high-fidelity region dives – this is a capacity limit, not a tuning one. We therefore **freeze the repaired base and train one bounded residual head per ball region** (right and left $\times$ low, mid, up). Each head (`GoalkeeperBallisticResidual`) is zero-initialized – so training starts exactly at the strong base – and receives explicit *ballistic timing features*: the time and (y, z) of the ball at the keeper plane $x = 0$ and at the goal plane $x = -0.5$, the incoming v_x , and the ball speed. Trained with the stable save-window collection of Stage C and a region focus, each specialist (`stable_sr{region}.pt`) masters its own balls without the cross-region trade-off that caps the single network. Because the residual is bounded and zero-initialized, a specialist can only *refine* the base dive for its region, never destabilize it.

3.5 Stage E — Mixture of experts with an analytic ballistic gate

At inference the six specialists are combined into a single deployable bundle (`bundle_moe6.py` $\rightarrow$ `goalkeeper_moe6_hard3_default.pt`) and routed by an **analytic ballistic-crossing gate** – no learned parameters and no privileged information. From the raw ball position and velocity the gate extrapolates, in closed form, the ball’s state when it reaches the keeper plane: the crossing time t , the lateral crossing c_y , the crossing height c_z , and the descent rate v_z at crossing. Thresholds map c_z to a height band ($c_z < 0.85$ m $\rightarrow$ low, $c_z > 1.35$ m $\rightarrow$ up, else mid) and $c_y < 0$ to the left side, selecting one of the six experts. The region is **latched early** – as soon as the ball is unambiguously approaching ($v_x < -1$ and within 5 m of the keeper) – so the correct specialist drives the *entire* dive rather than a default expert mishandling the opening frames (which is fatal for fast balls to the far side). Finally, an optional **left/right symmetry mirror** serves a weak corner through its stronger mirrored expert (`mirror_obs` in, `mirror_action` out, FK-validated in `symmetry.py`), correcting the teacher’s residual left-bias. This mixture is what lifts the keeper above both the single repaired network and the $\approx 72.5\%$ teacher.

4 Phase 1 result and analysis

Evaluation protocol. All numbers use the official grader script, `scripts/eval_naive_goalkeeper.py` on the Eval-Goalkeeper task – a 6-region parabolic ball, success defined exactly as the rubric requires (the ball does not cross the goal plane inside the frame), with `fell_over` termination disabled so a diving save that ends on the ground still counts. The deliverable is the MoE6 bundle; the script auto-detects it and rebuilds the six experts plus the gate (Section 6). Reproduce with:

```
MUJOCO_GL=egl python scripts/eval_naive_goalkeeper.py \
--headless --num-trials 50 \
--checkpoint src/assets/soccer/weight/goalkeeper_moe6_hard3_default.pt
```

Block rate. The deployed mixture reaches a best-observed $475/500 = 95.0\%$ on the official headless eval and a more stable $\approx 93.8\%$ over 16,384 trials; we treat the latter as the trustworthy estimate, since the 500-trial figure is the best of a higher-variance protocol. This sits above the 91% sibling MoE on the `keeper-qbs` branch, the 81.7% single repaired network (Stage C), and the $\approx 72.5\%$ reference teacher of Ren et al. [2] on the identical benchmark – and is obtained without an unrealistic robot model and without deploying the released checkpoint as-is. Under the rubric’s linear band – zero at 80%, full marks at 100% – 93.8% maps to a score of $(0.938 - 0.80)/0.20 \times 30 \approx 20.7/30$ (22.5/30 at the best-observed 95%). **[TODO: measure the official 50-trial grader integer on the exact deployed bundle. `goalkeeper_moe6_hard3_default.pt` is not committed to the repository (only the MoE6 code and the base checkpoints are), so the figures above are PR #10’s reported large-trial evals; they still need a local 50-trial re-measurement on the deployed checkpoint, which is steep and variance-sensitive near the band.]**

Residual failure structure. The floor is set by the genuinely hard high balls: the upper corners trail the near-solved low and mid regions, and the remaining misses are *timing*-limited near-misses – the nearest limb is a few centimeters short at the crossing instant – rather than reach or routing failures. Most gate misroutes go to a same-side-adjacent expert and are still saved, so routing accuracy is not the binding constraint; the early latch (Stage E) is what removed the dominant error, by preventing a default expert from committing fast far-side balls in the wrong direction during the opening frames.

Why a mixture rather than a single network. The single repaired network of Stage C reaches 81.7% and, tellingly, its weakest region shifts between runs: improving one corner regresses the other – the signature of a capacity ceiling rather than a tuning problem. The mixture removes exactly this ceiling – each specialist is freed from the cross-region trade-off – which is why the deployed bundle clears 90% where every single-network variant we trained did not. The per-scenario repair oracle of Stage B reaches an even higher save rate *per ball*, but that is a per-scenario optimum, not deployable headroom (replaying its open-loop dives at inference scores *below* the base on any other ball); the deployable gain comes from the specialists and the gate, not from executing oracle trajectories directly.

5 Secondary experiments and negative results

Three findings around the mixture are reported honestly: one clean negative that shaped the specialist training, one variance caution, and one in-progress path to push past the current best.

On-policy ballistic-residual PPO (negative). Before training the specialists from oracle data, we tried the obvious alternative: freeze the base and learn each bounded, zero-initialized residual head directly

with PPO [3] on the block reward. The runs *stayed* near the frozen base and produced no clear gain – the conservative residual is safe but does not, on its own, synthesize the hard-ball saves. This negative is exactly why the Stage D specialists are trained by distilling the repair-oracle demonstrations rather than by on-policy reward: the productive signal lives in the oracle, not in on-policy exploration of the residual.

Gate-threshold sweeping (variance caution). The analytic gate has a few scalar thresholds (the height bounds and the latch distance), and sweeping them (`sweep_moe6_gate.py`) over short runs produces tempting candidates. We found these unreliable: a candidate that looked best on a few hundred trials regressed under a large-trial re-evaluation. We therefore fix the gate thresholds and trust only the 16,384-trial estimate, not short gate sweeps.

Failure mining and failure-replay PPO (in progress). To push past the current $\approx 93.8\%$, the branch includes a pipeline that mines the mixture’s residual misses, replays those specific failing balls, and fine-tunes the responsible specialist on them (`collect_moe6_failures.py`, `run_failure_replay_pipeline.sh`). It is included as the next-stage path but is **not** the selected deployed checkpoint, so we do not claim its gains here.

6 Phase 2: deployment behind the competition API

For the cross-evaluation tournament the policy is served behind the standardized `/act` and `/reset` HTTP API (`scripts/api_server.py`). The grader’s `compete.py` sends the **same raw MuJoCo state** to every team – root pose/velocity, joint positions/velocities and last action for both robots, plus the ball position and velocity – and the server turns that raw state into exactly the observation tensor the policy was trained on. The match between this server-side observation and the training observation is what the entire mixture depends on, so we specify it concretely.

Per-frame observation (96-D), computed server-side from raw state. On each `/act` call `compute_goalkeeper_obs` builds one 96-D frame by concatenating six terms, each transformed into the goalkeeper’s own pelvis frame with the exact scalings used in training (Table 1). Only the ball position and the keeper’s own proprioception are used; no privileged or opponent state enters the observation. The key non-obvious detail is that the reference frame and the per-term scalings (0.25, 0.05, the GK default joint offset) are goalkeeper-specific and **must** match the training config – a wrong scale or a world-frame ball vector silently destroys the policy at deployment even though the network loads fine.

Table 1: Goalkeeper per-frame observation (96-D) built from raw state.

Term	Dim	Computation from raw state
<code>ball_pos_local</code>	3	ball position relative to pelvis: <code>quat_apply(quat_inv(root_quat), ball_pos - root_pos)</code>
<code>base_ang_vel</code>	3	root angular velocity rotated into body frame, scaled $\times 0.25$ (the GK PD-gain ratio)
<code>projected_gravity</code>	3	gravity $(0, 0, -1)$ rotated into body frame via <code>quat_apply(quat_inv(root_quat), g)</code>
<code>joint_pos_rel</code>	29	<code>joint_pos - GK_default_joint_pos</code> (scale $\times 1.0$)
<code>joint_vel_scaled</code>	29	<code>joint_vel</code> $\times 0.05$
<code>last_action</code>	29	the previous action returned by the server

Term-major history stacking (→ 960-D). The actor consumes a 10-frame history, but **not** as a naive frame-major concatenation. `stack_goalkeeper_history` reproduces mjlabs’ *term-major* layout: it concatenates all 10 frames of `ball_pos_local` first, then all 10 frames of `base_ang_vel`, and so on per term, yielding the $96 \times 10 = 960$ -D vector the MLP expects. The server keeps a 10-slot history buffer; on the first frame after a reset it pre-fills all 10 slots with the initial frame so the very first action is well-defined.

Endpoints and MoE6 routing. `/act` computes the 96-D frame, updates the history, term-major-stacks it to 960-D, and runs the policy under `inference_mode`. Because the analytic gate needs ground-truth ball state, it also forwards the raw ball position and velocity via `set_raw_ball_state` before the forward pass; the gate then extrapolates the crossing point, latches the region for the episode, and the latched specialist produces the action. `/reset` clears the per-episode latches, each expert’s state, and the history buffer. On load, `_load_policy` detects the moe6 bundle key and rebuilds `GoalkeeperMoE6Policy` – the six ballistic-residual experts (from the bundle’s `sr` list, each itself a frozen base plus a residual head) and the gate constants – from the single checkpoint, so the same server code also serves the plain MLP, single-residual, and recurrent variants unchanged.

Deployment status (integration gap). The MoE6-capable server above is `scripts/api_server.py` on the keeper branch. The integrated `phase2/api_server.py` on main is at present the generic template – a single-frame (~ 96 -D) observation with no history stacking and no moe6 detection – and would *not* serve the mixture correctly. Wiring the MoE6 server and bundle into the `phase2/` tournament setup is therefore an outstanding step. **[TODO: confirm which `api_server` and checkpoint the team actually deploys for the Phase 2 tournament, and wire the MoE6 keeper into `phase2/`.]**

Part B Shooter

7 Overview

Task. The G1 humanoid (29 DoF) performs a penalty-spot shot in MuJoCo: approach a stationary ball and kick it into the goal. This is a perception-guided motor skill requiring a motion-tracking prior and ball-trajectory control. Table 2 summarizes the deliverables.

Table 2: Shooter deliverables.

Artifact	Description
Six-stage progressive training pipeline	Stage 1 → Stage 6, 20K iterations, ≈ 39 B steps
Stage-6 model	LSTM 128-64-32 + 2×128 , actor 160-D / critic 298-D
Multi-strategy API server	<code>uniform</code> / <code>best-motion</code> / <code>gk-aware</code> shooting strategies
Evaluation tooling	per-motion analysis, kick-timing analysis, parallel batched eval
Performance matrix	per-motion accuracy/speed data over 250K trials

8 Training pipeline: six-stage progressive training

Following the progressive perception–action framework of PAiD [1], we design six training stages. All stages use the same LSTM architecture; capability is grown purely through the environment configuration (rewards, terminations, coordinate system), summarized in Table 3.

Table 3: The six training stages.

Stage	Goal	Coordinates	Key new rewards
1	Motion tracking (imitate kick motions)	Training	six <code>track_body</code> terms, pure imitation
2	Perception-guided kicking	Training	<code>contact</code> + <code>sideways_kick</code> + <code>ball_vel_align</code>
3	Goal-plane accuracy + speed curriculum	Training	<code>goal_accuracy</code> + <code>goal_miss</code>
4	High-speed accuracy (target 10 m/s)	Training	<code>is_terminated</code> = -500, <code>ball_speed</code> = 20
5	Competition-frame alignment + full-goal accuracy	Competition	<code>goal_accuracy</code> = 30, <code>ball_oob</code> = -30
6	High-speed kicking in competition frame	Competition	<code>ball_speed</code> = 40, <code>z_speed</code> = -5

8.1 Coordinate systems

- **Training frame (Stages 1–4):** robot near the origin, goal along $-y$. `motion_origin_offset` + `motion_yaw_offset` map motion-local coordinates into world coordinates.
- **Competition frame (Stages 5–6):** fully aligned with `compete.py` – robot at (4, 0, 0.8) facing $-x$, ball at (3, 0, 0.1), goal plane $x = -0.5$, width 3.0 m, height 1.8 m.

8.2 Training environment parameters

Table 4: Training environment parameters.

Parameter	Value
Parallel environments	4096
Steps per environment	24
Control frequency	50 Hz (decimation = 4, timestep = 0.005 s)
Episode length	10.0 s
Action space	29-D joint position targets
Action scaling	G1_ACTION_SCALE (per-joint PD gains)
PPO learning rate	1×10^{-3} (adaptive schedule)
Discount γ	0.99
GAE λ	0.95
KL target	0.01
Entropy coef.	0.005
GPU	2× NVIDIA GeForce RTX 4090 D (24 GB)
Training time (Stage 6)	≈ 9 h
Total steps (Stage 6)	$\approx 3.93 \times 10^9$

8.3 Per-stage detail

Stage 1 — Motion tracking. Learn to imitate diverse kick-motion references with no perception input, forming a stable motion prior. *Command sampling:* adaptive (biased toward high-failure motion segments; kernel size 3, $\lambda = 0.1$, uniform ratio 0.1). *Domain randomization:* `push_robot` (random base-velocity impulse every 1–3 s), observation noise (± 0.01 to ± 0.5 per term), observation corruption. *Terminations* (5): `time_out`; `fell_over` (base tilt $> 70^\circ$); `anchor_pos_z` (z error > 0.25 m); `anchor_ori` (orientation error > 0.8); `ee_body_pos` (end-effector position error > 0.25 m). The last three are motion-reference guards.

Stage 2 — Perception-guided kicking. Introduce ball perception and soccer kick rewards; relax motion constraints to allow ball chasing. Command sampling switches from adaptive to uniform (each episode

starts at frame 0). New rewards: `proximity` (1.0), `contact` (50.0, foot-ball contact with horizontal force), `sideways_kick` (50.0, ball moves toward goal), `ball_vel_align` (30.0), `ball_speed` (10.0), `foot_distance` (0.2), `is_terminated` (−200), and cheat suppressors `both_feet_ball` (−5), `nonfoot_ball` (−3), `foot_stomp` (−20), `foot_lift` (−2). Motion-tracking weight `anchor_pos` drops to 0.0 (free chasing); `track_foot_pos`= 1.0 added. `ee_body_pos` threshold relaxed 0.25 → 0.35 m.

Stage 3 — Goal-plane accuracy + speed curriculum. Add an explicit goal-plane crossing reward and adaptive target sampling. `ball_speed`= 15 (std widened to 3.0); **new** `goal_accuracy`= 10 (Gaussian, std= 0.3); **new** `goal_miss`= −5 (ball never reaches the goal plane); the six motion-tracking terms halve 1.0 → 0.5. Terminations are sharply pruned to just `time_out` + `fell_over` (all motion-reference guards removed). Adaptive target: 11 goal-plane bins, $\alpha = 0.3$ (focus on the hardest-to-hit regions).

Stage 4 — High-speed accuracy. Push ball speed toward ≈ 10 m/s. `ball_speed`= 20 (std 5.0); `is_terminated` −200 → −500 (a fall is more costly at speed); `undesired_contacts` −0.1 → −1.0 (prevent body dragging); `goal_miss` −5 → −20.

Stage 5 — Competition-frame alignment. Key innovation: switch the training frame entirely to `compete.py`’s world coordinates, eliminating the train→compete domain gap – the model now sees exactly the match scene during training. `goal_accuracy` 10 → 30; `goal_miss`→`goal_miss_scaled` (−50, scaled by lateral miss distance); `ball_vel_align`= 30 (std tightened to 0.12); `z_speed` re-enabled (−2, mild anti-lob); **new** `ball_oob`= −30 (ball leaves the goal bounds); **new** `both_feet_air_time`= −5 (both feet airborne > 0.15 s); `proximity` 1.0 → 3.0; the six motion-tracking terms drop to ≈ 0.05 each (total 0.3). *Domain randomization: fully off* – zero perturbation, noise, corruption, or push – a pure competition environment.

Stage 6 — High-speed competition. Push ball speed toward 12–16 m/s in the competition frame. `ball_speed` 20 → **40** (std 10.0); `z_speed` −2 → **−5** (anti-lob at speed); `ball_vel_align` 30 → 20 (std relaxed to 0.20); `goal_accuracy` 30 → 20 – trading accuracy for speed.

9 Model architecture and observation space

9.1 Network

```
RNNModel (Actor & Critic share the architecture):
  MLP encoder: Linear(128->128) -> ELU -> Linear(128->64) -> ELU -> Linear(64->32) -> ELU
  RNN:         LSTM(input_dim, hidden=128, num_layers=2)
  Output head: Linear(32->output_dim)

Actor:  input_dim=160, output_dim=29 (joint position targets)
Critic: input_dim=298, output_dim=1  (value)
```

Action distribution: `GaussianDistribution` with a scalar (single learnable log-std), initial std = 1.0.
 Observation normalization: `EmpiricalNormalization` on both actor and critic.

9.2 Actor observation space (160-D)

All terms are normalized by their per-term std before concatenation.

Table 5: Actor observation terms (160-D).

#	Term	Dim	Description
0	command	58	reference joint positions (29) + velocities (29) from the motion command
1	projected_gravity	3	gravity vector in the base frame
2	motion_ref_ang_vel	3	reference torso angular velocity (world frame)
3	base_ang_vel	3	measured base angular velocity (IMU gyro)
4	joint_pos	29	current joint position – HOME_KEYFRAME default
5	joint_vel	29	current joint velocity
6	actions	29	previous-step action output
7	target_point_pos	3	ball position in the robot pelvis frame
8	target_destination_pos	3	shot target point in the pelvis frame

9.3 Critic privileged information (+138-D)

These five extra terms are available only during training (privileged information), giving the critic ground-truth state for more accurate value estimation: `motion_anchor_pos_b` (3), `motion_anchor_ori_b` (6, 6-D rotation), `body_pos` (42, 14 tracked bodies in anchor frame), `body_ori` (84, 14 bodies, 6-D rotation), `base_lin_vel` (3, world frame).

9.4 Domain randomization schedule

Domain randomization contracts monotonically from Stage 1 to Stage 6 (Table 6). The fully randomization-free Stage 5–6 design ensures exact consistency with the `compete.py` environment.

Table 6: Domain-randomization schedule.

Stage	Sampling	Push robot	Obs noise	Obs corruption	Ball randomization
1	adaptive (failure-biased)	yes	yes (actor)	yes	—
2–4	uniform	yes	yes (actor)	yes	curve offset ± 0.15 m
5–6	uniform	no	no	no	no

9.5 Motion files

Ten `soccer-standard-*.npz` motion files are used: six right-leg kicks (`001/004/005/006/009/010_right`) and four left-leg kicks (`002/003/007/008_left`). Each file stores joint positions (T , 29), joint velocities (T , 29), and torso angular velocity (T , 3); lengths 205–289 frames (4.1–5.8 s).

10 Reward design evolution

There are 28 reward terms at Stage 6, but the core signal is driven by a few high-weight terms. The design intent is a gradual transition from *pure imitation* to *pure match performance* (Table 7).

Regularization (shared across stages). `action_rate` (-0.1 , L_2 on action delta); `joint_limit` (-10.0); `undesired_contacts` ($-0.1/-1.0$, non-foot / non-wrist ground contact).

Table 7: Core reward-term evolution across stages.

Reward term	S1	S2	S3	S4	S5	S6	Function
track_body_pos	1.0	1.0	0.5	0.5	0.05	0.05	body pose tracking
track_body_ori	1.0	1.0	0.5	0.5	0.05	0.05	body orientation tracking
track_anchor_ori	1.0	1.0	0.5	0.5	0.05	0.05	anchor orientation tracking
track_body_lin_vel	1.0	1.0	0.5	0.5	0.05	0.05	body linear-velocity tracking
track_body_ang_vel	1.0	1.0	0.5	0.5	0.05	0.05	body angular-velocity tracking
track_foot_pos	—	1.0	0.5	0.5	0.05	0.05	foot position tracking
tracking total	6.0	5.0	3.0	3.0	0.3	0.3	
contact	—	50	50	50	50	50	foot-ball contact force
sideways_kick	—	50	50	50	50	50	lateral kick toward goal
ball_vel_align	—	30	30	30	30	20	ball velocity aligned to goal
ball_speed	—	10	15	20	20	40	ball horizontal speed
goal_accuracy	—	—	10	10	30	20	goal-plane crossing accuracy
goal_miss	—	—	−5	−20	−50	−50	miss penalty
z_speed	—	0	0	0	−2	−5	vertical-speed penalty
ball_oob	—	—	—	—	−30	−30	out-of-bounds penalty

Design logic. (1) *Stage 1 (pure imitation)*: six tracking terms total 6.0, no soccer reward, forming a stable kick prior. (2) *Stage 2 (add soccer)*: sparse rewards (contact= 50, sideways_kick= 50) drive foot-ball interaction while negative terms suppress cheating; tracking eased (anchor_pos→ 0). (3) *Stages 3–4 (accuracy + speed)*: dense goal-accuracy reward added, ball_speed climbs, goal_miss hardens −5 → −20 → −50, tracking halved (total 3.0), terminations pruned 5 → 2. (4) *Stages 5–6 (competition alignment + extreme speed)*: tracking near zero (0.3), ball_speed doubles 20 → 40 to become the main signal, z_speed hardens to −5, accuracy signals slightly lowered (ball_vel_align 30 → 20, goal_accuracy 30 → 20) to trade accuracy for speed.

11 Phase 1 evaluation

11.1 Setup

Official grader eval_naive_shooter.py (single-env, task Eval-Shooter) and a parallel eval_shooter_parallel.py (task Eval-Shooter-Stage6, 4096 envs). Model: Stage 6 model_20000.pt. Seeds: 42, 2810, 202606. Trials/seed: 50 (naive) / 10000 (parallel). Score formula $\max(0, (\text{success} - 0.8)/0.2) \times 30$.

11.2 Results

Table 8: Shooter Phase-1 results (parallel eval, 10000 trials/seed).

Seed	Trials	Goals	Success rate
42	10000	9996	99.96%
2810	10000	9995	99.95%
202606	10000	9997	99.97%
Mean	10000	9996.0	99.96%

Phase-1 score: $\max(0, (0.9996 - 0.8)/0.2) \times 30 = 29.9/30$. Representative metrics (seed 202606): mean kick speed **13.03** m/s, kick accuracy (cosine similarity) 0.9787 ± 0.0191 . The goal rate exceeds 99.9% at all three seeds (only 3–5 misses per 10000); the 0.1-point shortfall from full marks is the linear map magnifying the tiny gap from 100%.

11.3 Evaluation tooling

To meet the fixed-seed requirement we hardened the eval scripts: (1) four-level seed control (`random / np.random / torch.manual_seed / env_cfg.seed`); (2) a `-seeds` argument running multiple seeds and auto-aggregating the mean and Phase-1 score; (3) per-trial `policy.reset()` to clear LSTM hidden state; (4) explicit GPU memory management between seeds (`del + gc.collect() + torch.cuda.empty_cache()`); (5) a parallel evaluator giving $\approx 25\times$ speedup over 4096 envs with identical goal-detection logic.

12 Phase 2: competition strategy and API server

12.1 API server

A FastAPI server loads the Stage-6 LSTM policy (task Eval-Shooter-Stage6) and exposes `POST /act` (raw state $\rightarrow$ obs $\rightarrow$ inference $\rightarrow$ action) and `POST /reset` (clear hidden state, choose the shot target by strategy). **Key decision:** the server computes its own observation (160-D `compute_shooter_obs`) from raw MuJoCo state, so the observation space is fully private to us – `compete.py` passes only raw state and each team computes its own observation, enabling interoperability across heterogeneous systems.

Observation computation (`compute_shooter_obs`). From the raw JSON state, extract the shooter’s 29 joint positions/velocities and last action, base orientation and angular velocity, and the ball world position; transform the ball and destination positions into the pelvis frame; read the reference joint positions/velocities and torso angular velocity from the motion file at the corresponding timestep; concatenate the nine terms into the 160-D observation.

12.2 Three shooting strategies

Table 9: Shooting strategies.

Strategy	Trigger	Behavior	Latency
uniform	<code>/reset</code>	random motion (incl. left leg) + random target $y \in [-1.4, 1.4]$, locked immediately	none
best-motion	<code>/reset</code>	random target + <code>_select_motion_for_dest()</code> picks the best right-leg motion, locked immediately	none
gk-aware	per-frame <code>/act</code>	observe the GK for 5–16 frames, then lock the far-side target + best motion	5–16 frames

12.3 The gk-aware strategy

This is the default and most capable strategy, in three phases.

Observation phase (`shoot_planned["done"]==False`): each `/act` appends the GK root-y to `gk_y_history`; `_should_lock()` applies a dual condition – *behavioral trigger* (GK lateral offset ≥ 0.3 m for a run of `min_observe=5` frames $\rightarrow$ early lock) and *hard deadline* (reaching `deadline_pct=60%` of the kick frame $\rightarrow$ force lock). Crucially we do **not** reset the policy on lock: locking only changes the `dest_body` observation term and leaves the LSTM hidden state intact, because the policy was trained on a continuous motion stream and jumping to frame 16 with an empty hidden state causes an out-of-distribution collapse.

Target planning (`_plan_destination`): if the GK is left ($y < -0.3$), shoot right `random(0.3, 1.4)`; if right ($y > +0.3$), shoot left `random(-1.4, -0.3)`; if central ($|y| \leq 0.3$), shoot the full goal width `random(-1.4, 1.4)`. The random component within the chosen half adds unpredictability.

Execution: `_select_motion_for_dest(dest_y, motions)` selects the best right-leg motion; the target world coordinate is set to `(-0.5, dest_y, 0.11)` (goal-plane center); the motion continues to play and the policy steers the kick from the target term.

12.4 Target region $\rightarrow$ motion map

Built from `eval_motion_analysis.py` (5000 trials per motion $\times$ target):

Table 10: Target region to best motion (data-driven).

Target y range	Best motion	Speed	Accuracy	Reason
$[-1.5, -0.7]$ far left	<code>006_right</code>	16.2 m/s	0.10 m	best far-corner accuracy
$[-0.7, -0.3]$ mid-left	<code>009_right</code>	14.2 m/s	0.35 m	best right-leg mid-left accuracy
$[-0.3, +0.3]$ center	<code>010_right</code>	16.4 m/s	0.11 m	highest speed + good center accuracy
$[+0.3, +0.7]$ mid-right	<code>005_right</code>	12.1 m/s	0.006 m	best accuracy overall (sub-cm)
$[+0.7, +1.5]$ far right	<code>005_right</code>	15.7 m/s	0.64 m	most versatile motion

All selected motions are right-leg kicks. Left-leg motions are unused because they are slower (6–9 m/s) and completely fail the far-left corner (0% goal rate).

12.5 Kick-frame map and zero-GK baseline

`analyze_kick_timing.py` confirms all motions contact the ball at frames 26–30 (0.52–0.60 s) with zero std, which makes the `_should_lock()` deadline reliable (e.g. `001_right` kicks at frame 27/260, `005_right` at 26/230, `006_right` at 27/225). When `compete.py` omits `-goalkeeper-api`, the GK is a `ZeroPolicy` (holds the `HOME_KEYFRAME` crouch at $y = 0$), giving a pure technical baseline.

13 Motion performance analysis

Scale: 10 motions $\times$ 5 targets $\times$ 5000 trials = **250,000** trials, model Stage 6 (`model_200000.pt`), targets $y = -1.50, -0.75, 0.00, +0.75, +1.50$ (even split of the 3.0 m goal).

Overall. Mean goal rate **94%** (9/10 motions at 100%); kick trigger rate 100%; right-leg mean ball speed 14.2 m/s (≈ 51 km/h); left-leg 8.0 m/s (≈ 29 km/h); right-leg mean accuracy 0.42 m; left-leg 0.26 m.

Table 11: Right-leg motion performance (all 100% goal rate).

Motion	Frames	Mean speed	Mean accuracy	Hit@0.3m	Best target	Best accuracy
<code>001_right</code>	260	13.59 m/s	0.340 m	40%	+0.75	0.152 m
<code>004_right</code>	257	14.02 m/s	0.548 m	20%	+0.75	0.197 m
<code>005_right</code>	230	13.96 m/s	0.402 m	20%	+0.75	0.006 m
<code>006_right</code>	225	14.56 m/s	0.343 m	40%	-1.50	0.095 m
<code>009_right</code>	205	13.73 m/s	0.454 m	40%	+0.00	0.144 m
<code>010_right</code>	230	15.40 m/s	0.512 m	20%	+0.00	0.112 m

Table 12: Left-leg motion performance (80% goal rate; 0% only at $y = -1.50$).

Motion	Frames	Mean speed	Mean accuracy	Hit@0.3m
002_left	214	7.27 m/s	0.245 m	80%
003_left	289	7.88 m/s	0.277 m	65%
007_left	229	8.42 m/s	0.266 m	60%
008_left	225	8.36 m/s	0.261 m	60%

Table 13: Target $\times$ ball-speed matrix (m/s), right-leg motions.

Motion	$y=-1.50$	$y=-0.75$	$y=+0.00$	$y=+0.75$	$y=+1.50$
001_right	14.58	14.04	14.14	12.41	12.80
004_right	13.02	14.52	14.62	12.53	15.41
005_right	14.59	13.37	14.12	12.06	15.69
006_right	16.18	14.18	14.04	12.99	15.42
009_right	14.40	14.24	12.14	12.54	15.33
010_right	14.64	14.57	16.42	16.28	15.10

Key findings. (1) *Strong right/left asymmetry:* right-leg ball speed is $\approx 2\times$ the left, and the left leg cannot reach the far-left corner – likely because right-leg motions are more numerous/better in the data or the reward structure favors the right leg. (2) *Far-corner accuracy degrades:* all motions lose accuracy at $y = \pm 1.50$ (0.4–1.2 m vs. 0.1–0.4 m at center), a reasonable mechanical limit. (3) *Competition recommendation:* use right-leg motions only, with the `gk-aware` strategy that reads the GK during execution and shoots to the far side.

14 Engineering decisions and lessons

Unified architecture. All six stages share an identical LSTM (128-64-32 + 2×128), differing only by environment config. This lets checkpoints transfer seamlessly across stages (`-resume` loads actor+critic+normalizer), concentrates effort on reward/environment design rather than re-tuning networks, and speeds iteration (changing a reward weight needs no network redesign).

Coordinate alignment is a fatal detail. Stage 5 switching the training frame fully to `compete.py`’s world coordinates (robot (4, 0, 0.8), yaw π) looks trivial but is decisive: it removes the sim-to-sim domain gap. Checkpoints trained earlier in the training frame degrade markedly in the compete scene.

LSTM state continuity. Not calling `policy.reset()` on target lock in `gk-aware` was a key lesson. An initial implementation reset on `should_lock`, feeding the LSTM an empty hidden state at motion frame 16 – a combination never seen in training – causing an OOD collapse. The fix updates only the `dest_body` term and preserves LSTM continuity.

Data-driven motion selection. The `_select_motion_for_dest()` map is not heuristic but grounded in the 250K-trial measurements; each target region’s recommendation is backed by concrete speed/accuracy numbers, avoiding subjective guessing.

Rule-based vs. adversarial training. Phase 2 pits the shooter against an unknown opponent GK. Rather than *adversarial training* (which is unstable when two policies learn online and would compound an already

complex six-stage curriculum), we chose a *strong single agent + rule-based target selection*: first train a near-perfect GK-free shooter (99.96% goal rate), then add a **gk-aware** rule that reads the GK and shoots to the far side. “Shoot to the far side” is a physically deterministic decision that needs no learning, and given a shooter that scores 12–16 m/s anywhere in the goal, this beats a random target by a wide margin against a reactive GK. The trade-off is reduced adaptivity to complex GK behavior (occlusion, tackles), but interpretability and controllability are preferable against an unknown opponent to a possibly under-trained adversarial policy.

References

- [1] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, “Learning soccer skills for humanoid robots: A progressive perception-action framework,” 2026. arXiv:2602.05310.
- [2] J. Ren, J. Long, T. Huang, H. Wang, Z. Wang, F. Jia, W. Zhang, J. Wang, P. Luo, and J. Pang, “Humanoid goalkeeper: Learning from position-conditioned task-motion constraints,” arXiv:2510.18002, 2025.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [4] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. AISTATS*, 2011, pp. 627–635.
- [5] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, and G. Martius, “Sample-efficient cross-entropy method for real-time planning,” in *Proc. CoRL*, PMLR vol. 155, 2021, pp. 1049–1065.
- [6] R. Y. Rubinstein and D. P. Kroese, *The Cross-Entropy Method*. New York: Springer, 2004.
- [7] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033.
- [8] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, “Learning to walk in minutes using massively parallel deep reinforcement learning,” in *Proc. CoRL*, PMLR vol. 164, 2022, pp. 91–100.